

Lab 10: Socket Programming 2

Duration: 2 hours | In Lab Evaluation & Assignment

Tools: Python 3 (`socket`, `selectors`)

Objectives

By the end of this lab, students will be able to:

- Implement a simple length-prefixed messaging protocol over TCP and explain why framing is necessary.
- Build a minimalist HTTP-like server using raw TCP sockets and observe how HTTP uses TCP.
- Implement a reliable file transfer on top of UDP (stop-and-wait with sequence numbers, ACKs, timeouts, retransmission).
- Compare performance (throughput and latency) of TCP and UDP-based transfers under loss and delay using Python timing.
- Use non-blocking I/O and `selectors` (`select/poll`) to serve multiple TCP clients without threads.
- Demonstrate and tune socket options: `SO_REUSEADDR`, `TCP_NODELAY`, `SO_RCVBUF`/`SO_SNDBUF`.

Lab Description

Part 0: Tools & Quick Checks

1. Verify Python 3 is installed:

```
python3 --version
pip3 --version
```

2. Install any extra modules (if missing):

```
pip3 install --user pyOpenSSL
```

Part A: TCP Framing : Length-Prefixed Messages

Why Framing is Required in TCP TCP is a **byte-stream protocol**, not a message-oriented protocol. This means:

- TCP guarantees ordered and reliable delivery of bytes.
- TCP does *not* preserve message boundaries.
- A single `send()` may be received in multiple `recv()` calls.
- Multiple sends may be merged into one receive.

Therefore, applications must implement their own **message framing mechanism** to determine where one message ends and the next begins.

In this implementation, we use a **length-prefixed protocol**, where:

[4-byte length] [message payload]

The first 4 bytes (unsigned integer, big-endian) specify how many bytes follow. This guarantees correct reconstruction of messages regardless of TCP segmentation.

Server (length-prefixed receive loop) — tcp_frame_server.py

```

1 import socket
2 import struct
3
4 # Listen on all network interfaces
5 HOST = '0.0.0.0'
6 PORT = 7007
7
8 def recv_all(sock, n):
9     """
10    Receive exactly n bytes from the socket.
11    TCP does not guarantee that recv(n) returns n bytes,
12    so we must loop until all bytes are received.
13    """
14    data = b''
15    while len(data) < n:
16        chunk = sock.recv(n - len(data))
17        if not chunk: # Connection closed
18            return None
19        data += chunk
20    return data
21
22 # Create TCP socket (IPv4, Stream-based)
23 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
24
25 # Allow quick reuse of the port after restart
26 server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
27
28 # Bind to address and port
29 server.bind((HOST, PORT))
30
31 # Listen for incoming connections (queue size 5)
32 server.listen(5)
33 print("Length-prefixed TCP server on", PORT)
34
35 # Accept one client connection
36 conn, addr = server.accept()
37 print("Connected by", addr)
38
39 try:
40     while True:
41         # Step 1: Read the 4-byte length header
42         hdr = recv_all(conn, 4)
43         if not hdr:
44             break # Client disconnected
45
46         # '!I' = Network byte order (big-endian) unsigned int
47         (length,) = struct.unpack('!I', hdr)
48
49         # Step 2: Read exactly 'length' bytes of payload

```

```
50     payload = recv_all(conn, length)
51     if payload is None:
52         break
53
54     print("Got message:", payload.decode())
55
56     # Step 3: Echo back using same framing format
57     # First pack length, then append payload
58     conn.sendall(struct.pack('!I', len(payload)) + payload)
59
60 finally:
61     conn.close()
62     server.close()
```

Client (send framed messages) — tcp_frame_client.py

```
1  import socket
2  import struct
3  import sys
4
5  # Use command-line argument if provided
6  HOST = sys.argv[1] if len(sys.argv) > 1 else '127.0.0.1'
7  PORT = 7007
8
9  # Create TCP socket
10 client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
11
12 # Connect to server
13 client.connect((HOST, PORT))
14
15 for msg in ["hello", "this is a longer message", "bye"]:
16     # Convert string to bytes
17     data = msg.encode()
18
19     # Send: 4-byte length prefix + actual data
20     client.sendall(struct.pack('!I', len(data)) + data)
21
22     # Receive 4-byte response header
23     hdr = client.recv(4)
24     if not hdr:
25         print("Server closed")
26         break
27
28     # Extract message length
29     (length,) = struct.unpack('!I', hdr)
30
31     # Receive full response payload
32     resp = b''
33     while len(resp) < length:
34         chunk = client.recv(length - len(resp))
35         if not chunk:
36             break
37         resp += chunk
```

```
38
39     print("Echo:", resp.decode())
40
41 client.close()
```

What to Observe

- If a large payload is sent, `recv()` may return only part of the data.
- TCP may split a single message into multiple packets.
- TCP may combine multiple small messages into one packet.
- Without length-prefixing, the receiver cannot determine message boundaries.

Why Length-Prefixing Solves the Problem

By sending the message size first:

1. The receiver knows exactly how many bytes belong to the message.
2. The receiver keeps reading until all bytes are received.
3. Message boundaries are reconstructed correctly.

This technique is widely used in real-world protocols such as RPC systems, database wire protocols, and binary application-layer protocols.

Part B: Build a Minimal HTTP-like Server Using TCP

This task implements a very small HTTP/1.0-style server using TCP sockets. The purpose is to demonstrate how application-layer protocols (such as HTTP) run on top of TCP, which provides reliable and ordered byte-stream delivery.

Unlike earlier examples where we manually framed messages, HTTP defines its own message format consisting of:

- A request line (e.g., `GET /index.html HTTP/1.1`)
- Headers
- A blank line (`\r\n\r\n`)
- Optional message body

The server below handles basic GET requests and serves files from a local directory.

How to Run the Server in a Browser

1. Create the following folder structure:

```
project/
  mini_http.py
  www/
    index.html
```

2. Inside `www/index.html`, add:

```
<h1>Hello from my minimal HTTP server</h1>
```

3. Run the server:

```
python mini_http.py
```

4. Open your browser and go to:

```
http://localhost:8080
```

The browser sends an HTTP request over TCP to port 8080. The server processes the request and returns the file contents.

Simple HTTP Server — mini_http.py

```

1 import socket
2 import os
3
4 # Listen on all local interfaces, port 8080
5 HOST, PORT = '0.0.0.0', 8080
6
7 # Directory where files are served from
8 WEBROOT = './www'
9
10 def handle(conn):
11     """
12     Handle one client HTTP request.
13     Reads until the end of HTTP headers (\r\n\r\n),
14     parses the request line, and serves a file.
15     """
16     req = b''
17
18     # Read until end of HTTP headers
19     # HTTP headers end with a blank line: \r\n\r\n
20     while b'\r\n\r\n' not in req:
21         chunk = conn.recv(1024)
22         if not chunk:
23             return # Client closed connection
24         req += chunk
25
26     # Extract first request line (e.g., GET /index.html HTTP/1.1)
27     first_line = req.split(b'\r\n', 1)[0].decode()
28
29     try:
30         method, path, version = first_line.split()
31     except ValueError:
32         return # Malformed request
33
34     # Remove leading slash from requested path
35     path = path.lstrip('/')
36
37     # Default file is index.html
38     if path == '':
39         path = 'index.html'
40
41     # Construct full file path inside WEBROOT
42     full_path = os.path.join(WEBROOT, path)

```

```
43
44     # Check if requested file exists
45     if os.path.isfile(full_path):
46         with open(full_path, 'rb') as f:
47             body = f.read()
48
49         # Build minimal HTTP/1.0 success response
50         response = (
51             b"HTTP/1.0 200 OK\r\n"
52             + b"Content-Length: " + str(len(body)).encode() + b"\r\n"
53             + b"\r\n"
54             + body
55         )
56     else:
57         # File not found response
58         response = (
59             b"HTTP/1.0 404 Not Found\r\n"
60             + b"Content-Length: 0\r\n"
61             + b"\r\n"
62         )
63
64     # Send full response over TCP
65     conn.sendall(response)
66
67
68     # Create TCP socket (IPv4, Stream)
69     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
70
71     # Allow quick restart of server
72     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
73
74     # Bind to host and port
75     server.bind((HOST, PORT))
76
77     # Listen for incoming connections
78     server.listen(5)
79
80     print("Mini HTTP server running on http://localhost:8080")
81
82     # Main server loop (sequential handling)
83     while True:
84         conn, addr = server.accept()
85         try:
86             handle(conn)
87         finally:
88             conn.close()
```

What This Demonstrates

- HTTP is an application-layer protocol built on top of TCP.
- TCP provides reliable, ordered byte delivery.

- The application (HTTP) defines how messages are structured.
- The browser acts as an HTTP client communicating over TCP.

Limitations of This Minimal Implementation

- Only supports GET requests.
- No persistent HTTP/1.1 connections.
- No MIME type headers.
- Handles one client at a time (sequential).

Despite its simplicity, this example clearly shows how web servers use TCP sockets to implement higher-level application protocols.

Part C: Reliable File Transfer Over UDP

Create a simple reliable transfer over UDP using sequence numbers and ACKs. This shows how an application can add reliability over an unreliable transport.

Protocol outline

- Sender reads file and splits into chunks (e.g., 1024 bytes).
- Each datagram = 4-byte sequence number (big-endian) + payload.
- Receiver replies with 4-byte ACK = highest in-order seq received.
- Sender uses timeout (e.g., 0.5s) and retransmits chunk on timeout.
- Transfer completes when sender sends special sequence number -1 (0xFFFFFFFF).

Send snippet (sender) : udp_file_sender.py

```

1 import socket
2 import struct
3 import time
4 import sys
5
6 DEST = (sys.argv[1], int(sys.argv[2]))
7 fname = sys.argv[3]
8 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
9 sock.settimeout(0.5)
10
11 seq = 0
12 with open(fname, 'rb') as f:
13     while True:
14         chunk = f.read(1024)
15         if not chunk:
16             break
17         pkt = struct.pack('!I', seq) + chunk
18         while True:
19             sock.sendto(pkt, DEST)
20             try:
21                 data, _ = sock.recvfrom(8)
22                 (ack,) = struct.unpack('!I', data[:4])

```

```

23         if ack == seq:
24             seq += 1
25             break
26         except socket.timeout:
27             print("Timeout, retransmit seq", seq)
28
29     # send end marker
30     sock.sendto(struct.pack('!I', 0xFFFFFFFF), DEST)
31     sock.close()

```

Receive snippet (receiver) : udp_file_receiver.py

```

1  import socket
2  import struct
3  import sys
4
5  PORT = int(sys.argv[1])
6  out = sys.argv[2]
7  sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
8  sock.bind(('0.0.0.0', PORT))
9  with open(out, 'wb') as fo:
10     expected = 0
11     while True:
12         data, addr = sock.recvfrom(2048)
13         (seq,) = struct.unpack('!I', data[:4])
14         if seq == 0xFFFFFFFF:
15             print("Transfer complete")
16             # optionally ACK the end marker
17             sock.sendto(struct.pack('!I', 0xFFFFFFFF), addr)
18             break
19         payload = data[4:]
20         if seq == expected:
21             fo.write(payload)
22             sock.sendto(struct.pack('!I', seq), addr)
23             expected += 1
24         else:
25             # resend ACK for last in-order (expected-1) to prompt
26             # retransmit
27             last = (expected - 1) if expected > 0 else 0xFFFFFFFF
28             sock.sendto(struct.pack('!I', last), addr)
29     sock.close()

```

Part D: Non-blocking I/O and selectors

Serve multiple TCP clients without creating a thread per client using selectors.

Selector-based echo server skeleton — sel_echo.py

```

1  import selectors
2  import socket
3
4  sel = selectors.DefaultSelector()
5
6  def accept(sock):

```

```
7     conn, addr = sock.accept()
8     conn.setblocking(False)
9     sel.register(conn, selectors.EVENT_READ, read_conn)
10
11 def read_conn(conn):
12     data = conn.recv(1024)
13     if data:
14         conn.sendall(data) # echo
15     else:
16         sel.unregister(conn)
17         conn.close()
18
19 sock = socket.socket()
20 sock.bind(('0.0.0.0', 9009))
21 sock.listen()
22 sock.setblocking(False)
23 sel.register(sock, selectors.EVENT_READ, accept)
24
25 print("Selector-based server on 9009")
26 while True:
27     events = sel.select()
28     for key, _ in events:
29         callback = key.data
30         callback(key.fileobj)
```

Exercise: Connect multiple clients and observe single-threaded multiplexing.

Part E: Socket Options and Tuning

Demonstrate and experiment with options:

- `SO_REUSEADDR` — allow quick rebinding after a crash (shown in example servers).
- `TCP_NODELAY` — disable Nagle's algorithm to reduce latency for many small writes.
`sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)`
- `SO_SNDBUF` / `SO_RCVBUF` — inspect and set buffer sizes:
`sock.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, 262144)`
`print(sock.getsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF))`
- `SO_LINGER` (brief explanation) — change close behavior to wait for unsent data.